



PROJETO FINAL

Descrição do problema

Nos últimos anos, os sistemas de compartilhamento de bicicletas se tornaram uma alternativa sustentável e prática para a mobilidade urbana em diversas cidades ao redor do mundo. Seguindo essa tendência, a Prefeitura de João Pessoa anunciou a implantação do sistema *JP-Bike*, com o objetivo de oferecer aos cidadãos e turistas uma opção de transporte que reduza o tráfego, diminua a emissão de poluentes e incentive hábitos mais saudáveis.

No entanto, para que o sistema funcione de forma eficiente, não basta apenas instalar as estações e disponibilizar as bicicletas. É necessário também garantir que elas estejam distribuídas adequadamente enquanto o sistema está em funcionamento. Ao longo do dia, certas estações podem sofrer escassez de bicicletas, enquanto outras acumulam excesso, prejudicando a experiência dos usuários. Esse desafio é conhecido como rebalanceamento do sistema e deve ser realizado periodicamente para assegurar o seu correto funcionamento.

O rebalanceamento consiste em planejar rotas para veículos que transportarão bicicletas entre as estações, de modo a corrigir os desequilíbrios. Os veículos podem partir com um número inicial de bicicletas, visitar estações que precisam receber ou doar bicicletas, e retornar ao depósito ao final de suas rotas. Para que a operação seja eficiente, é preciso minimizar o custo total das rotas, respeitar a capacidade de carga dos veículos e garantir que todas as demandas das estações sejam atendidas.

O sucesso desse planejamento é fundamental para manter a confiabilidade do *JP-Bike*, aumentando a satisfação dos usuários e garantindo que o sistema seja sustentável tanto do ponto de vista operacional quanto econômico.

Formalmente, o problema pode ser descrito da seguinte forma:

Seja $G = (V, A)$ um dígrafo completo, onde o conjunto de vértices $V = \{0, 1, \dots, n\}$ é particionado em um depósito (vértice 0) e estações (vértices $1, 2, \dots, n$). Cada estação i possui uma demanda q_i , que pode ser **positiva** (estações de coleta, das quais q_i bicicletas devem ser removidas) ou **negativa** (estações de entrega, às quais $|q_i|$ bicicletas devem ser entregues). Uma frota de m veículos idênticos, cada um com capacidade Q , está disponível no depósito para transportar as bicicletas. Existe um custo de viagem c_{ij} associado a cada arco $(i, j) \in A$.

O problema consiste em determinar até m rotas, cada uma iniciando e terminando no depósito. Ao longo de cada rota, o número de bicicletas transportadas nunca deve ser negativo nem exceder a capacidade Q . Cada estação deve ser visitada exatamente uma vez, e sua demanda deve ser completamente atendida pelo veículo que a visita. Bicicletas retiradas de estações de coleta podem ser entregues tanto a estações de entrega quanto devolvidas ao depósito, e bicicletas entregues a estações de entrega podem ter origem no depósito ou em estações de



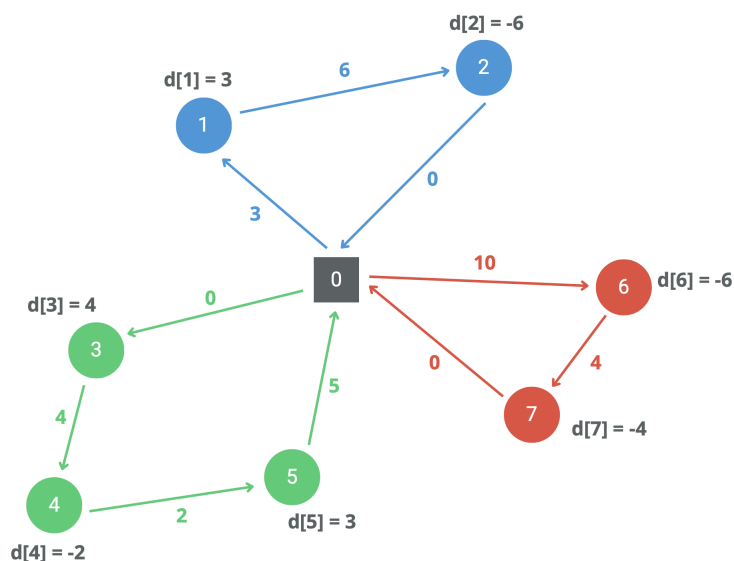
coleta. Se necessário, é permitido que um veículo parta do depósito já carregado com um determinado número de bicicletas e também retorne carregado ao depósito. O objetivo é encontrar o conjunto de rotas que respeite todas essas restrições e minimize o custo total, dado pela soma das distâncias percorridas por cada veículo.

Exemplo de instância e solução

Para exemplificar o problema, considere uma instância (cenário) com um total de $n = 7$ estações (sem incluir o depósito) e uma frota com $m = 4$ veículos, cada um com capacidade $Q = 10$. As demandas dos clientes são dadas por $d = \{3, -6, 4, -2, 3, -6, -4\}$. A matriz c contendo o custo (distância) de cada arco é como a seguir.

c_{ij}	0	1	2	3	4	5	6	7
0	-	6	10	25	21	20	20	20
1	4	-	21	5	20	30	20	27
2	9	26	-	20	4	27	26	17
3	30	6	15	-	25	16	10	7
4	16	21	5	24	-	14	7	11
5	14	30	24	15	14	-	22	19
6	20	20	29	14	9	20	-	21
7	15	27	17	9	13	20	20	-

Considerando essa instância, o seguinte planejamento é uma solução viável (não necessariamente a ótima) para o problema. Note que, o valor indicado em cada arco representa a carga total do respectivo veículo ao trafegar por aquele trecho.





Algumas observações importantes sobre essa solução:

- Somente 3 dos 4 veículos foram utilizados, o que é perfeitamente viável;
- Em nenhuma rota a capacidade dos veículos foi ultrapassada, pois a carga permaneceu dentro do intervalo $[0, 10]$ durante todo o percurso;
- Para a rota do veículo vermelho, foi necessário partir do depósito com uma carga inicial de 10 bicicletas, a fim de atender às demandas dos clientes visitados. Observa-se que, **para essa rota**, o valor pode ser obtido diretamente pela soma das demandas dos clientes na rota: $d_6 + d_7 = -6 - 4 = -10$, indicando que são necessárias 10 bicicletas adicionais para viabilizar o atendimento.
- No caso do veículo verde, a soma das demandas da rota é igual a $d_3 + d_4 + d_5 = 4 - 2 + 3 = 5$. Esse resultado indica a existência de um excedente de 5 bicicletas na rota, o qual, conforme ilustrado na figura, é devolvido ao depósito;
- O valor da solução é dado pela soma do custo de cada rota, ou seja:

$$\text{Custo total} = \underbrace{c_{01} + c_{12} + c_{20}}_{\text{rota azul}} + \underbrace{c_{03} + c_{34} + c_{45} + c_{50}}_{\text{rota verde}} + \underbrace{c_{06} + c_{67} + c_{70}}_{\text{rota vermelha}} = 36 + 78 + 56 = \mathbf{170}$$

É importante destacar que, de modo geral, a soma das demandas associadas a uma rota específica o seu desbalanceamento. Quando essa soma é positiva, o veículo retorna ao depósito transportando bicicletas excedentes. De forma análoga, quando a soma é negativa, o veículo deve partir do depósito com uma carga inicial para suprir as demandas ao longo do percurso. Entretanto, deve-se ressaltar que a viabilidade de uma rota não pode ser avaliada apenas com base no fato de o depósito poder prover bicicletas adicionais ou absorver o excedente. Por exemplo, considere a seguinte rota para um dos veículos da instância do exemplo anterior:

$$0 \rightarrow 6 \rightarrow 2 \rightarrow 1 \rightarrow 0$$

A soma das demandas dessa rota é $d_6 + d_2 + d_1 = -6 - 6 + 3 = -9$, o que, a princípio, parece sugerir que o veículo deve partir do depósito com um total de 9 bicicletas. Entretanto, caso isso aconteça, o veículo chegará ao cliente 2 apenas com 3 bicicletas e não conseguirá satisfazer a demanda de tal cliente. Observe ainda que, mesmo que o veículo iniciasse a rota com sua capacidade máxima (10 bicicletas), a sequência permaneceria inviável. Portanto, nem toda sequência de visitas é viável, mesmo que a soma das demandas pertença ao intervalo $[-Q, Q]$.

Instruções

O projeto deve ser realizado em grupo de **3 integrantes (da mesma turma)** e vale 10 pontos, relativos à terceira nota da disciplina. Cada grupo deve desenvolver um algoritmo eficiente de busca local (ou meta-heurística) para o problema de otimização descrito acima. O código-fonte deve ser **obrigatoriamente** escrito na linguagem C/C++. **Não é permitido** a utilização de ferramentas como ChatGPT e similares para auxiliar no desenvolvimento do código.



Note que o seu programa deve ser capaz de ler um arquivo contendo os dados de uma instância (cenário) do problema e utilizar tais dados como entrada para o algoritmo. O formato de arquivo a ser utilizado é o seguinte:

```
1 n
2 m
3 Q
4
5 array d
6
7 matriz c
```

A instância utilizada na seção anterior, por exemplo, poderia ser representada pelo seguinte arquivo:

```
1 7
2 4
3 10
4
5 3 -6 4 -2 3 -6 -4
6
7 0 6 10 25 21 20 20 20
8 4 0 21 5 20 30 20 27
9 9 26 0 20 4 27 26 17
10 30 6 15 0 25 16 10 7
11 16 21 5 24 0 14 7 11
12 14 30 24 15 14 0 22 19
13 20 20 29 14 9 20 0 21
14 15 27 17 9 13 20 20 0
```

Ao final da execução, **seu código deve produzir um arquivo de saída**, no seguinte formato, contendo a melhor solução encontrada para cada instância de teste:

```
1 <valor da solucao>
2 <numero de veiculos utilizados>
3 <rota do veiculo 1>
4 <rota do veiculo 2>
5 ...
6 <rota do ultimo veiculo utilizado>
```

Por exemplo, a solução mostrada na seção anterior geraria o seguinte arquivo de saída:

```
1 170
2 3
3 0 1 2 0
4 0 3 4 5 0
5 0 6 7 0
```

Etapas e prazos

Este projeto contém os seguintes entregáveis:

- Implementação de ao menos um algoritmo guloso para a geração de uma solução inicial viável;



- Implementação de pelo menos 3 movimentos de vizinhança;
- Implementação do algoritmo de busca local chamado VND (Variable Neighborhood Descent);
- Implementação de uma meta-heurística (OPCIONAL). Sugestões: GRASP ou ILS;
- Resultados computacionais: **criar uma tabela** que contenha os resultados obtidos pela(s) heurística(s) construtiva(s) e pelo VND, e que compare tais resultados com a solução ótima de cada instância. Essa tabela deverá conter os seguintes dados para cada heurística construtiva e para o VND:
 - Média do valor da solução (em no **mínimo 10 execuções** para cada instância caso exista algum fator aleatório no algoritmo);
 - Melhor solução encontrada;
 - Média do tempo gasto pelo respectivo algoritmo;
 - GAP para a solução ótima.

Observação: Caso decida implementar a meta-heurística, é necessário adicionar colunas de resultados para ela na tabela;

- Todas as implementações devem vir acompanhadas de um arquivo *makefile* para a compilação. Tal arquivo deve ser preparado de forma a funcionar em sistemas UNIX;
- Criar uma pasta contendo os arquivos de saída gerados durante os testes com cada instância. Favor incluir somente os resultados dos testes finais, com a versão a ser entregue.

O projeto deve ser entregue até as **23:59 do dia 17 de setembro de 2025**. Devem ser enviados, via SIGAA, o código-fonte do projeto e um relatório em *pdf* contendo o nome dos integrantes do grupo e a tabela de resultados computacionais. Note que é necessário somente uma entrega por grupo e **não serão aceitos envios por e-mail ou fora do prazo**.

Avaliação

Cada grupo deverá apresentar presencialmente o projeto em data a ser agendada pelo professor. A nota do projeto é individual e leva em consideração diversos critérios, como demonstração de entendimento do código na apresentação, qualidade do código, eficiência dos algoritmos implementados, qualidade dos resultados obtidos, dentre outros. **Não apresentar o projeto implica nota zero.**

Dicas

Como calcular o valor da medida GAP: Suponha que desejamos calcular o valor GAP para o resultado da heurística construtiva para a instância chamada *nome_instancia*. Supondo que o valor encontrado pela heurística para essa instância é dado por $valor_{heuristica}$ e o valor ótimo para essa instância é $valor_{otimo}$, o cálculo do GAP é realizado da seguinte forma:



$$gap = \left(\frac{valor_{heuristica} - valor_{otimo}}{valor_{otimo}} \right) \times 100$$

O valor do gap é dado em porcentagem (%) e indica a “distância” da solução, no caso, da heurística construtiva, para o valor ótimo. Note que, para calcular o GAP dos resultados obtidos pelo VND, basta substituir $valor_{heuristica}$ pelo valor encontrado pelo VND.

Exemplo de tabela de resultados:

	ótimo	Heurística construtiva			VND		
		valor solução	tempo	gap	valor solução	tempo	gap
instancia1	0.0	0.0	0.0	0.0	0.0	0.0	0.0
instancia2	0.0	0.0	0.0	0.0	0.0	0.0	0.0
instancia3	0.0	0.0	0.0	0.0	0.0	0.0	0.0
instancia4	0.0	0.0	0.0	0.0	0.0	0.0	0.0